PARKEASE — SMART PARKING OPERATIONS SYSTEM

STUDENT PROJECT WORK DIARY / PROGRESS LOG

An IoT-Enabled, Multi-Zone Automated Parking Operations Platform with Computer Vision ANPR

Project Title:	ParkEase: IoT-Enabled Smart Parking Management System	**Academic Term:**	Final Year Project (2026–2027)
Team Lead:	Abhishek (Team Leader)	**Department:**	Computer Science & Engineering
Project Members:	Abhishek & Student Project Team	**Supervisor/Guide:**	Project Faculty Supervisor
Tech Stack:	Python, Flask, MySQL 8.0, Three.js, OpenCV, EasyOCR, IoT/MQTT	**Diary Duration:**	8-Day Intensive Development Sprint

DAY 1 — System Architecture Planning, Requirements & Repository Setup

Time Logged: 7.5 hrs

Target Objective:	Define project scope, architecture diagram, user role matrix, and dev environment.
Work Executed:	• Analyzed urban parking pain points: congestion, manual billing delays, and lost vehicle disputes. • Engineered the multi-tier system architecture: Flask backend, MySQL 8.0 database, Three.js visualizer, and OpenCV/EasyOCR pipeline. • Configured Git version control, Python 3 virtual environment, directory layout, and security configs (.env). • Created formal system requirements specification (SRS) and modular sprint plan for the 8-day milestone.
Tools & Tech:	Python, Git, Markdown, Architecture Design Tools
Key Outcome:	**SRS document approved; baseline project scaffolding created and committed to repository.**
Supervisor Sign-off:	Remarks: [ ] Satisfactory [ ] Needs Revision **Guide Signature:** _____

DAY 2 — MySQL 8.0 Relational Schema Engineering & Model Persistence

Time Logged: 8.0 hrs

Target Objective:	Implement normalized relational database schema with data integrity constraints.
Work Executed:	• Engineered 10 normalized tables in MySQL: users, vehicles, parking_slots, parking_sessions, pricing_rules, payments, hardware_devices, recognition_logs, device_events, and audit_logs. • Configured strict integrity constraints: foreign keys, generated columns (active_vehicle_key, active_slot_key) to prevent double-occupancy race conditions. • Mapped SQLAlchemy ORM models with relationship properties and transaction management context managers. • Created SQL initialization scripts (01_users.sql, 02_parkease_schema.sql, 03_seed_development.sql) and verified connections.
Tools & Tech:	MySQL 8.0, PyMySQL, SQLAlchemy, Database Normalization
Key Outcome:	**Database schema initialized on port 3306 with sample slots, pricing rules, and admin credentials.**
Supervisor Sign-off:	Remarks: [ ] Satisfactory [ ] Needs Revision **Guide Signature:** _____

DAY 3 — Core Parking Business Logic, State Machine & Exit Calculations

Time Logged: 8.5 hrs

Target Objective:	Build entry/exit workflows, rate calculation engine, and contract test suite.
Work Executed:	• Implemented services/parking.py for atomic check-in, registration normalization, and bay assignment. • Built services/pricing.py supporting deterministic hourly billing, minimum fees, 15-minute grace period, and EV charging kWh tariffs. • Engineered services/exit_workflow.py for timestamp normalization, 2-second immediate exit tolerance, and cash receipt generation. • Wrote automated pytest suite (test_parking_contract.py, test_exit_workflow_contract.py) achieving 100% test pass rate.
Tools & Tech:	Python, Flask, Pytest, PyMySQL, Decimal arithmetic
Key Outcome:	**Core ticketing and billing service fully functional and verified with 17 automated tests.**
Supervisor Sign-off:	Remarks: [ ] Satisfactory [ ] Needs Revision **Guide Signature:** _____

DAY 4 — Computer Vision ANPR Pipeline & Indian RTO Decoder

Time Logged: 8.0 hrs

Target Objective:	Integrate optical character recognition for license plates and state jurisdiction decoding.
Work Executed:	• Integrated EasyOCR and OpenCV (cv2) for automated vehicle license plate recognition from camera images. • Added position-aware OCR correction for Indian registration plates (e.g. converting lookalikes O/0, I/1). • Engineered services/rto_decoder.py: comprehensive database decoding all 36 Indian States/UTs, RTO offices (DL01–DL14, MH01–MH14, KA01–KA05), and Bharat Series (22BH). • Implemented bounding box calculation [x, y, w, h] to highlight detected plates on visual feeds.
Tools & Tech:	OpenCV, EasyOCR, NumPy, Regular Expressions, Python
Key Outcome:	**Automated plate detection operational with Indian RTO decoding and confidence scoring.**
Supervisor Sign-off:	Remarks: [ ] Satisfactory [ ] Needs Revision **Guide Signature:** _____

DAY 5 — Interactive 3D Digital Twin Lot Map (Three.js WebGL) & Camera Presets

Time Logged: 8.5 hrs

Target Objective:	Build live 3D visual representation of parking lot with real-time bay statuses.
Work Executed:	• Engineered 3D parking lot canvas using Three.js with WebGL renderer, perspective camera, and OrbitControls. • Built Multi-Zone rendering: Zone A, Zone B, and Basement EV Hub with distinct neon shader colors (Cyan for EV, Gold for VIP, Green for Available, Orange for Occupied). • Added interactive Raycaster ray-tracing: clicking any 3D bay opens a modal showing active vehicle plate and session details. • Implemented camera preset controls (3D Isometric, Top-Down Aerial, Gate Cam, Reset) and HUD occupancy bar.
Tools & Tech:	JavaScript (ES Modules), Three.js, WebGL, CSS Glassmorphism
Key Outcome:	**Interactive 3D Digital Twin running smoothly at 60 FPS in browser with real-time slot state sync.**
Supervisor Sign-off:	Remarks: [ ] Satisfactory [ ] Needs Revision **Guide Signature:** _____

DAY 6 — Advanced Operations: Reservations, Dynamic Pricing UI & Review Queue

Time Logged: 8.0 hrs

Target Objective:	Develop operator interfaces for pre-booking, tariff adjustments, and low-confidence OCR review.
Work Executed:	• Engineered Advance Pre-Booking & Reservation portal (/reservations) backed by parking_reservations table with slot hold logic. • Built Dynamic Pricing Engine UI (/pricing): administrators can create custom tariffs, configure grace periods, EV rates, and activate rules with 1 click. • Created Human-in-the-Loop Review Queue (/anpr): OCR scans with confidence < 85% are queued for operator Accept, Correct, or Reject. • Built guided Lost Ticket Dispute Resolution workflow with flat penalty calculation and barrier override.
Tools & Tech:	Flask, Jinja2, HTML5/CSS3, JavaScript Fetch API
Key Outcome:	**Tariff management, reservation booking, and human review queue fully operational and integrated.**
Supervisor Sign-off:	Remarks: [ ] Satisfactory [ ] Needs Revision **Guide Signature:** _____

DAY 7 — Paperless Ticketing (Dynamic 2D QR Codes), WhatsApp & IoT Telemetry

Time Logged: 8.0 hrs

Target Objective:	Implement barcode ticketing, paperless driver messaging, and IoT hardware drivers.
Work Executed:	• Integrated Python qrcode to generate high-contrast 2D QR barcodes on entry tickets (ticket.html) and exit receipts (receipt.html). • Built Paperless Driver Messaging: 1-click WhatsApp E-Ticket and E-Receipt sharing via formatted API links. • Implemented IoT hardware device contracts (devices/base.py) and simulated boom barrier relay driver (devices/simulator.py). • Added live webcam feed integration on ANPR monitor with real-time green bounding box visualizer overlay.
Tools & Tech:	Python qrcode, WebRTC (getUserMedia), Canvas API, WhatsApp Web API
Key Outcome:	**Instant QR-code ticketing, WhatsApp paperless receipts, and live camera feed functional.**
Supervisor Sign-off:	Remarks: [ ] Satisfactory [ ] Needs Revision **Guide Signature:** _____

DAY 8 — Visual Analytics Dashboard, Cashier Settlement, Security & Final Audit

Time Logged: 8.5 hrs

Target Objective:	Complete analytical charts, shift handover settlement, CSV exports, and final QA verification.
Work Executed:	• Built Visual Analytics Dashboard (/analytics): 24-hour peak demand curve, 7-day revenue trend bar chart, and vehicle fleet distribution. • Built Cashier Shift Handover & Settlement Report (/settlement): today's shift totals (Cash vs UPI), cash drawer variance calculator, and printable slip. • Added 1-click RFC 4180 CSV export endpoints for sessions (/api/export/sessions.csv), payments, and administrative audit logs. • Implemented security controls: sliding-window IP rate limiter, CSRF token protection, and HTTP security headers. • Ran comprehensive end-to-end integration test: 19/19 pytest contract tests passing with 100% green status.
Tools & Tech:	Flask, SQL Aggregations, CSV Engine, Charting, Security Headers
Key Outcome:	**Complete, bug-free, enterprise-ready smart parking system successfully validated and documented.**
Supervisor Sign-off:	Remarks: [ ] Satisfactory [ ] Needs Revision **Guide Signature:** _____

FINAL PROJECT DIARY EVALUATION & SUPERVISOR ENDORSEMENT

Overall Project Assessment: The 8-day sprint progress submitted by the student team has been thoroughly reviewed and inspected against running system deliverables.	**Overall Evaluation Grade:** [ ] Excellent (A+) [ ] Good (A) [ ] Satisfactory (B)
Supervisor Cumulative Remarks: _____ _____ _____ _____	**Supervisor Signature with Date:** _____ Department Seal
Student Declaration: We hereby certify that the technical work, source code, and deliverables logged above represent authentic work completed by our project team.	**Team Lead Signature (Abhishek):** _____